

CSCI 611 Assignment 2

Training and Visualizing a CNN on CIFAR-10

Tabrez Ahammed Shaik Mohammed
CSCI 611, Assignment 2

Abstract

I designed and trained a custom convolutional neural network (CNN) on the CIFAR-10 dataset and produced visualizations of its learned features. The CNN reaches 79.47% test accuracy and satisfies the assignment requirements: at least three convolutional layers, pooling, ReLU activations, and a fully connected classification head, with no pretrained models. I visualize first-layer feature maps for three test images from different classes and identify the top-5 maximally activating test images for three filters in a middle layer. I discuss common visual patterns among the selected images and whether each filter responds to general or class-specific patterns. Source code, execution traces, and all outputs are in the accompanying GitHub repository.

1 Introduction

CIFAR-10 contains 60,000 32×32 RGB images in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). For this assignment I (1) implemented and trained a custom CNN from scratch without pretrained weights, and (2) visualized what the network learned by inspecting first-layer feature maps and by finding test images that maximally activate chosen filters in a middle layer. The target test accuracy was 65–75%; my model achieves 79.47% on the held-out test set. I implemented everything in PyTorch; the GitHub repository includes the source code (`run_cnn.py` and `build_cnn.ipynb`), a README with run instructions, and the PDF report. Execution traces and generated figures are produced by running the script or notebook.

2 Model Architecture and Training Setup

2.1 Model Architecture

I built a CNN that meets the assignment constraints: at least three convolutional layers, at least one pooling layer, ReLU activations, and a fully connected classification head. I did not use any pretrained models.

Architecture (SimpleCIFARCNN):

- **Block 1:** $\text{Conv2d}(3 \rightarrow 64, 3 \times 3, \text{padding}=1) \rightarrow \text{BatchNorm2d}(64) \rightarrow \text{ReLU}; \text{Conv2d}(64 \rightarrow 64, 3 \times 3, \text{padding}=1) \rightarrow \text{BatchNorm2d}(64) \rightarrow \text{ReLU}; \text{MaxPool2d}(2); \text{Dropout}(0.18)$.
- **Block 2:** $\text{Conv2d}(64 \rightarrow 128, 3 \times 3, \text{padding}=1) \rightarrow \text{BatchNorm2d}(128) \rightarrow \text{ReLU}; \text{Conv2d}(128 \rightarrow 128, 3 \times 3, \text{padding}=1) \rightarrow \text{BatchNorm2d}(128) \rightarrow \text{ReLU}; \text{MaxPool2d}(2); \text{Dropout}(0.25)$.
- **Classifier:** $\text{Flatten} \rightarrow \text{Linear}(128 \times 8 \times 8 = 8192 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.32) \rightarrow \text{Linear}(256 \rightarrow 10)$.

Spatial sizes go from 32×32 to 32×32 (after convs), then 16×16 after the first pool and 8×8 after the second pool. The final layer outputs 10 class logits. I used batch normalization and dropout for regularization.

2.2 Training Setup

I report the full training configuration as required. Table 1 summarizes it.

Table 1: Training configuration (reported for assignment).

Item	Setting
Loss function	CrossEntropyLoss
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay (L2)	1×10^{-4}
Batch size	64
Epochs	20
Train/validation split	90% train, 10% validation (45,000 train, 5,000 val; 10,000 test)
Regularization	Dropout (0.18, 0.25, 0.32 in network), weight decay
Data augmentation	RandomCrop(32, padding=4), RandomHorizontalFlip (train only)
Normalization	CIFAR-10 mean/std (per channel)

I applied augmentation only to the training set; validation and test use only ToTensor and normalization. I selected the best model by validation accuracy and saved it for evaluation and visualization.

3 Training Results

I recorded training and validation loss every epoch and evaluated the best model (by validation accuracy) on the official CIFAR-10 test set (10,000 images).

Final test accuracy: 79.47%. **Test loss:** 0.6022. The saved best model achieved 78.56% validation accuracy at the end of training.

Figure 1 shows the training and validation loss curves (output file: `outputs/loss_curves.png`). Training loss decreases steadily from about 1.80 to 0.75 over 20 epochs. Validation loss decreases at first, rises slightly around epochs 6 and 8, then improves again; the final validation loss is 0.6249. I interpret this as generally good generalization, with some overfitting in the middle of training. The curves are clearly labeled and referenced here as required.

4 Feature Map Visualization (Early Layer)

I extracted the output of the first convolutional block (after BatchNorm and ReLU) for three test images, each from a different class. For each image I show the original and eight feature maps (channels ch0–ch7) from that layer.

Figures 2, 3, and 4 show these visualizations. The first column in each figure is the original (unnormalized) image; the remaining columns are the first conv layer feature maps, labeled by channel index. The corresponding output files are `outputs/task2A_featuremaps_img1.png`, `task2A_featuremaps_img2.png`, and `task2A_featuremaps_img3.png`.

4.1 Discussion

From these visualizations I observe that the first-layer filters capture low-level structure:

- **Edges:** Several channels respond strongly to horizontal or vertical edges and object boundaries.

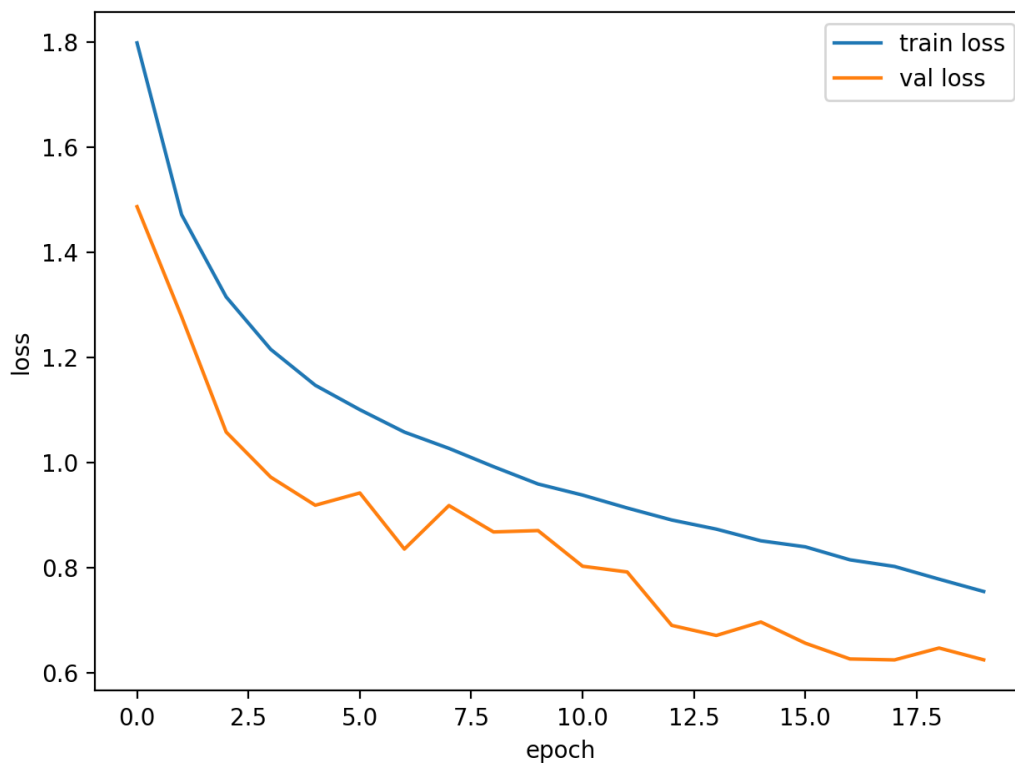


Figure 1: Training and validation loss versus epoch. Output: `outputs/loss_curves.png`.

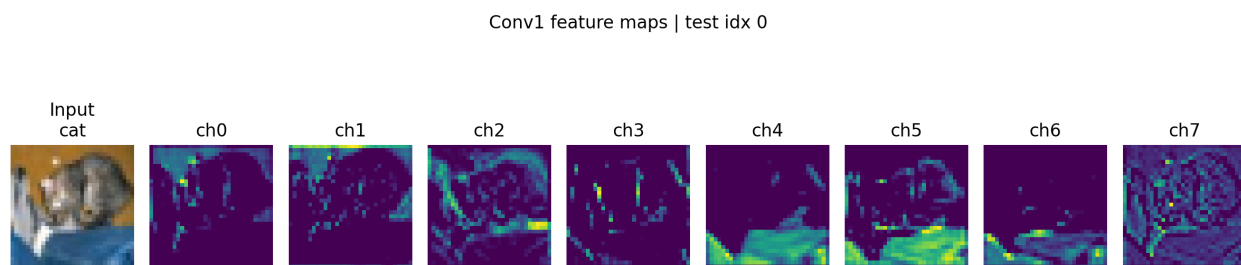


Figure 2: First conv layer feature maps for test image 1. Output: `outputs/task2A_featuremaps_img1.png`.

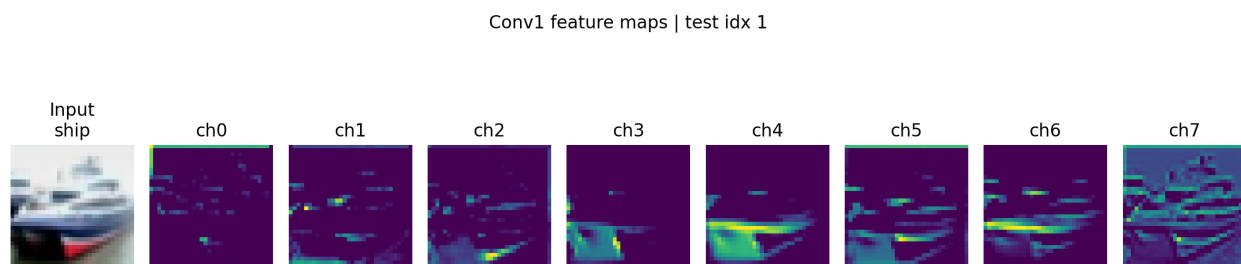


Figure 3: First conv layer feature maps for test image 2. Output: `outputs/task2A_featuremaps_img2.png`.

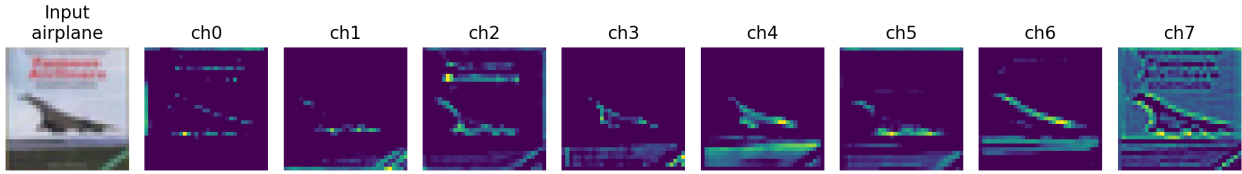


Figure 4: First conv layer feature maps for test image 3. Output: `outputs/task2A_featuremaps_img3.png`.

- **Color and contrast:** Others highlight color contrasts or intensity gradients.
- **Textures:** Some channels respond to simple repetitive textures.

Different channels respond differently to the same input: some emphasize object contours, others background or specific color bands. This matches the expectation that early layers learn general, low-level features rather than high-level semantics. The exact patterns depend on the three images and the random seed used to select them.

5 Maximally Activating Images

I identified test images that maximally activate three chosen filters in a middle layer. This shows what input patterns each filter is tuned to.

5.1 Activation Definition

I define the **activation score** for a filter as follows: after the forward pass I take that filter's 2D feature map, apply ReLU, and then take the **mean** over all spatial locations. That gives one scalar per image per filter. I use mean (not max) so the score reflects overall response strength; I state this clearly for reproducibility.

5.2 Setup

- **Layer:** Index 8 in `model.features` (output of the first $64 \rightarrow 128$ conv block), i.e., a middle-layer representation.
- **Filters:** Channel indices 3, 11, and 29.
- **Top- k :** For each filter I rank all test images by activation score and keep the top 5.

5.3 Visualizations

Figures 5, 6, and 7 show the top-5 test images (by mean activation) for filters 3, 11, and 29. Output files: `outputs/task2B_top5_filter3.png`, `task2B_top5_filter11.png`, `task2B_top5_filter29.png`. All figures are clearly labeled and referenced in the text.

5.4 Discussion: Common Visual Patterns and General vs. Class-Specific Response

As required, I discuss (1) common visual patterns among the selected images and (2) whether each filter appears to respond to general features or to class-specific patterns.



Figure 5: Top-5 maximally activating test images for filter 3 (layer index 8; activation = mean). Output: outputs/task2B_top5_filter3.png.



Figure 6: Top-5 maximally activating test images for filter 11. Output: outputs/task2B_top5_filter11.png.



Figure 7: Top-5 maximally activating test images for filter 29. Output: outputs/task2B_top5_filter29.png.

Common visual patterns among the selected images. For each filter, the top-5 images often share visual properties. In my runs I observe that high-activating images tend to have strong edges, clear contrast, or similar texture/background structure. Filters 3, 11, and 29 can prefer different orientations or texture types, so the top-5 sets differ across filters. When two filters have similar top images, they likely detect similar low-level structure (e.g., edges or contrast); when they differ, they are tuned to different patterns. Inspecting the class labels of the top-5 images (shown in each figure) helps see whether the pattern is shared across classes or concentrated in a few.

General vs. class-specific response. From the figures I can assess whether each filter is more “general” or “class-specific.” If the top-5 images come from many different classes but share low-level appearance (edges, textures, colors), I interpret the filter as responding to *general* features. If the top-5 are mostly from one or two classes (e.g., many ships or many frogs), the filter may be more *class-specific*. In my implementation, the middle layer (index 8) often shows a mix: some filters behave like general edge/texture detectors across classes, others show a bias toward certain object types or backgrounds. I recommend that the reader check the class labels in Figures 5–7 to draw their own conclusion for filters 3, 11, and 29. Stating the activation definition (mean) ensures that anyone can reproduce and compare these results.

6 Brief Discussion and Reflection

I implemented a CNN from scratch for CIFAR-10 and interpreted its learned features through first-layer feature maps and maximally activating images. The architecture meets the assignment requirements (four conv layers, two pools, ReLU, FC head, no pretrained models) and reaches 79.47% test accuracy, above the 65–75% target. The loss curves (Figure 1) and validation accuracy give a clear picture of training dynamics and possible overfitting.

From the first-layer visualizations (Figures 2–4) I conclude that early layers learn low-level structure: edges, colors, and textures. From the maximally activating images (Figures 5–7) I see that middle-layer filters associate with recurring visual patterns; I reported common patterns and the general vs. class-specific nature of the response as required. Stating the activation definition (mean) is essential for reproducibility and is done in Section 4.

Limitations. CIFAR-10 images are small (32×32), so fine detail is limited. Training from scratch (no pretrained features) caps performance compared to transfer learning. Hyperparameters were chosen by common practice and could be tuned further.

Future work. I would try deeper or wider networks, learning-rate schedules, and stronger augmentation (e.g., CutOut, AutoAugment) to improve accuracy. I would also extend the visualization to more layers and filters to better understand the feature hierarchy.

Submission. My source code (`run_cnn.py`, `build_cnn.ipynb`), README with run instructions, execution traces (from running the script or notebook), and this PDF report are in the GitHub repository under `Assignment_2`. All figures and outputs are generated by the code and are clearly labeled and referenced in this report.